

FleetLogix – Guía General del Proyecto

Introducción

FleetLogix es una empresa de transporte y logística que opera una flota de 200 vehículos realizando entregas de última milla en 5 ciudades principales. Actualmente utiliza sistemas legacy y hojas de cálculo, pero busca modernizar su infraestructura de datos para análisis en tiempo real y toma de decisiones basada en datos.

Objetivo del Proyecto Integrador (PI):

- Implementar y poblar una base de datos PostgreSQL robusta.
- Generar más de 500,000 registros sintéticos manteniendo integridad y coherencia.
- Documentar relaciones, constraints y métodos de generación de datos.
- Garantizar consistencia temporal y calidad de la información.

Avance 1 – PostgreSQL y Modelo Relacional

Base de Datos

Creación de la base:

```
CREATE DATABASE fleetlogix;
```

Este paso asegura que la base quede limpia y lista para recibir las tablas.

Usuario y permisos

```
CREATE USER fleetlogix_user WITH PASSWORD 'fleetlogix123';
```

```
ALTER DATABASE fleetlogix OWNER TO fleetlogix_user;
```

```
GRANT ALL PRIVILEGES ON DATABASE fleetlogix TO fleetlogix_user;
```

```
GRANT ALL PRIVILEGES ON DATABASE fleetlogix TO postgres;
```

Esto asegura que fleetlogix_user tenga control total, pero postgres también mantenga acceso.

Tablas del modelo relacional

Maestras

- **vehicles:** información de la flota (200 registros)
- **drivers:** información de conductores (400 registros)
- **routes:** rutas predefinidas (50 registros)

Transaccionales

- **trips:** viajes realizados (100,000 registros)
- **deliveries:** entregas individuales (400,000 registros)
- **maintenance:** historial de mantenimiento (5,000 registros)

Ejemplo SQL – Creación de vehicles

```
CREATE TABLE vehicles (  
  vehicle_id SERIAL PRIMARY KEY,  
  license_plate VARCHAR(20) UNIQUE NOT NULL,  
  vehicle_type VARCHAR(50) NOT NULL,  
  capacity_kg DECIMAL(10,2),  
  fuel_type VARCHAR(20),  
  acquisition_date DATE,  
  status VARCHAR(20) DEFAULT 'active'  
);
```

(Se incluyen tablas drivers, routes, trips, deliveries, maintenance con índices y comentarios como en el script completo.)

Generación de datos sintéticos

- Se utilizó Python con librerías **Faker** y **pandas**.
- Métodos clave:
 - generate_trips() → crea viajes históricos coherentes con vehículos, conductores y rutas.
 - gethourly_distribution() → asigna horas de salida realistas según patrón diario.
- Se mantiene la **integridad referencial** y la **consistencia temporal** (arrival > departure).

Control de Calidad

- Validación de integridad referencial completa.
- Verificación de consistencia temporal en trips y deliveries.
- Implementación de logs de carga de datos para seguimiento de errores.

Avance 2 – Consultas SQL y Optimización

Objetivo

Analizar problemas operativos reales de FleetLogix como:

- Eficiencia de rutas
- Carga de trabajo de conductores
- Mantenimiento de vehículos

Se deben ejecutar, documentar y optimizar **queries SQL complejas**, justificando qué problema de negocio resuelve cada una y mejorando su performance con índices.

Queries a Ejecutar

Se reciben **12 queries**, organizadas por nivel de complejidad:

Queries Básicas (3)

- Consultas simples con filtros y ordenamiento
- Joins básicos entre 2–3 tablas
- Agregaciones simples (COUNT, SUM, AVG)

Queries Intermedias (5)

- Joins múltiples (4–5 tablas)
- Subconsultas en WHERE y SELECT
- Agregaciones con GROUP BY y HAVING
- Funciones de fecha y string

Queries Complejas (4)

- CTEs múltiples y recursivas
- Window Functions avanzadas (RANK, LAG, LEAD)
- Subconsultas correlacionadas
- PIVOT / UNPIVOT operations
- Combinaciones de todas las técnicas anteriores

Selección de Queries para Entregar

- **Básicas:** ejecutar las queries 1, 2 y 3
- **Intermedias:** ejecutar las queries 4 y 5, y seleccionar **una** entre las queries 6, 7 o 8

- **Complejas:** ejecutar la query 9, y seleccionar **una** entre las queries 10, 11 o 12
- Total:** 8 queries a documentar en 02_queries_analysis.sql.

Documentación del Análisis

Para cada query, registrar en Manual_Consultas_SQL.pdf:

1. **Problema de negocio que resuelve**
2. **Tiempo de ejecución antes y después** de crear los índices
3. Breve explicación de **qué optimiza cada uno de los 5 índices** creados en 03_optimization_indexes.sql

Optimización

- Analizar cada query con EXPLAIN ANALYZE
- Crear 5 índices para mejorar el rendimiento basado en las queries ejecutadas
- Medir tiempos de ejecución **antes y después de optimizar**

Entregables

1. 02_queries_analysis.sql → 8 queries resueltas y optimizadas
2. 03_optimization_indexes.sql → 5 índices creados
3. Manual_Consultas_SQL.pdf → documentación del análisis, justificación de queries, tiempos de ejecución y explicación de índices

Recursos

- Acceso a las 12 queries: [Drive Link](#)

Avance 3 – ETL hacia Snowflake y Validación del Data Warehouse

Objetivo

- Diseño del ETL (05_ETL_Pipeline.py)
- Fases del Proceso (Extracción, Transformación, Carga)
- Validaciones y Control de Calidad
- Resultados y Métricas Finales
- Entregables

Diseño del ETL (05_ETL_Pipeline.py)

El pipeline ETL :

- Fue desarrollado en Python, utilizando librerías como pandas, sqlalchemy, y snowflake-connector-python.
- Implementa un flujo automatizado de extracción, transformación y carga (ETL) que conecta PostgreSQL con Snowflake.
- Su diseño modular permite escalabilidad y ejecución programada.

El script fue probado localmente y documenta cada fase del proceso mediante logs automáticos generados en el archivo etl_last_week.log.

Principales Componentes del ETL

- Extracción dinámica: se detecta automáticamente la última fecha cargada y se extraen los datos nuevos o modificados de los últimos 7 días.

- Transformación: los datos se limpian, estandarizan y estructuran en dimensiones (DIM_DATE, DIM_TIME, DIM_DRIVER, DIM_VEHICLE, DIM_ROUTE, DIM_CUSTOMER) y una tabla de hechos (FACT_DELIVERIES).
- Validaciones automáticas: se realizan controles de integridad referencial, duplicados, y coherencia temporal (por ejemplo, arrival_time > departure_time).
- Carga incremental: se utiliza la función write_pandas para insertar datos por lotes, con fallback a inserciones unitarias en caso de error de conexión o conflicto.

Validaciones y Control de Calidad

Antes de cargar la información en Snowflake, se implementaron pasos de verificación:

- Conteo de registros por tabla antes y después de la carga.
- Validación de claves foráneas entre tablas de hechos y dimensiones.
- Comparación de totales y conteos de entregas por día y ciudad.
- Log detallado de errores, advertencias y tiempos de ejecución.

Estas validaciones garantizan la consistencia entre el entorno origen y el destino, minimizando riesgos de pérdida de información o duplicidad en los procesos ETL.

Resultados y Validaciones Finales

Tras la ejecución completa del ETL, las tablas fueron correctamente cargadas en Snowflake. El log de ejecución confirmó la inserción de 3,889 registros en FACT_DELIVERIES y la actualización completa de todas las dimensiones.

Dimensiones cargadas:

- DIM_DATE (8 filas)
- DIM_TIME (1,246)
- DIM_DRIVER (400)
- DIM_VEHICLE (200)
- DIM_ROUTE (50)
- DIM_CUSTOMER (3,841)
- FACT_DELIVERIES (3,889)

Consultas de verificación confirmaron más de 3,700 clientes activos y más de 11,600 entregas totales. Asimismo, se validó la integridad de las relaciones entre hechos y dimensiones mediante claves compuestas y chequeos automáticos de consistencia temporal.

Entregables

1. 05_ETL_Pipeline.py → script completo de extracción, transformación y carga.
2. 06_snowflake_validation.sql → consultas de verificación y control de calidad en Snowflake.
3. Log_Ejecución_ETL.txt → registro detallado de la ejecución con métricas, errores y advertencias.

Avance 4 – Arquitectura Cloud en AWS (Serverless Pipeline)

Objetivo

En este avance, FleetLogix inicia la transición hacia una arquitectura **serverless en AWS**, con el propósito de procesar datos en tiempo real, eliminar dependencias locales y facilitar la escalabilidad del sistema

analítico.

El enfoque principal se basa en el diseño teórico y documentación de una infraestructura cloud modular, sin necesidad de desplegar los recursos efectivamente.

Diseño de Arquitectura

El entorno AWS se compone de los siguientes elementos:

- **API Gateway:** punto de entrada para las aplicaciones móviles de los conductores. Gestiona solicitudes HTTP POST hacia las funciones Lambda.
- **S3 (Amazon Simple Storage Service):** repositorio de datos históricos y logs del sistema, organizado por carpetas (raw-data/, processed-data/, backups/, logs/), con políticas de ciclo de vida hacia Glacier a los 90 días.
- **AWS Lambda:** funciones sin servidor que procesan los datos entrantes y ejecutan lógica operativa en tiempo real.
- **DynamoDB:** base NoSQL que almacena el estado actual de entregas, vehículos y rutas, permitiendo lecturas rápidas y concurrentes.
- **RDS PostgreSQL (AWS):** base de datos relacional administrada que replica la instancia local para análisis y persistencia histórica.
- **SNS (Simple Notification Service):** sistema de alertas que comunica desviaciones de ruta o incidencias en las entregas.

El diagrama de arquitectura (archivo aws_architecture_diagram.png) representa el flujo general de datos y eventos dentro del entorno AWS.

Componentes de Implementación

1. **06_aws_setup.py (modo teórico)**

Simula la creación y configuración de los recursos AWS mediante estructuras de texto y salidas de consola.

Incluye definiciones de:

- Buckets S3 y sus políticas de ciclo de vida.
- Tablas DynamoDB (deliveries_status, vehicle_tracking, routes_waypoints, alerts_history).
- Roles IAM para Lambda con permisos mínimos.
- Instancia teórica de RDS PostgreSQL con backups automáticos.
- Checklist de despliegue documentado.

Ejemplo de salida de simulación:

```
== Plan de Infraestructura (Simulación) ==
Región objetivo: us-east-1
Estructura S3 propuesta:
- s3://fleetlogix-dev-data/raw-data/
- s3://fleetlogix-dev-data/processed-data/
- s3://fleetlogix-dev-data/backups/
Tablas DynamoDB (teóricas):
- deliveries_status (PK: delivery_id)
- vehicle_tracking (PK: vehicle_id)
```

2. **lambda_functions.py (modo teórico)**

Define tres funciones Lambda principales, diseñadas para operar sobre los recursos de AWS:

- **Lambda 1 – lambda_verificar_entrega:**
Consulta la tabla deliveries_status en DynamoDB para determinar si una entrega fue

completada, devolviendo su estado y hora de confirmación.

Esta función se activa mediante **API Gateway** (POST /deliveries/verify).

- **Lambda 2 – lambda_calcular_eta:**

Calcula el tiempo estimado de llegada (ETA) de un vehículo en ruta, basándose en su ubicación actual y la del destino.

Registra los resultados en la tabla `vehicle_tracking` de DynamoDB y puede ejecutarse cada cinco minutos mediante **EventBridge**.

- **Lambda 3 – lambda_alerta_desvio:**

Detecta desviaciones de ruta mayores a 5 km utilizando los puntos de control (waypoints) de `routes_waypoints`.

Si se detecta un desvío, publica un mensaje de alerta en **SNS** y almacena el evento en `alerts_history`.

Cada Lambda está acompañada de bloques de configuración que describen sus *triggers* y permisos requeridos (DynamoDBFullAccess, SNSFullAccess).

Ejecución y Validación Teórica

El flujo simulado del pipeline en AWS fue probado en modo texto, validando que:

- Los recursos y tablas se definen correctamente.
- Las funciones Lambda incluyen manejo de excepciones, validaciones de entrada y escritura de logs.
- El script `06_aws_setup_MODO_TEÓRICO.py` produce la salida esperada sin conexión real a AWS.

No se ejecutaron comandos AWS CLI, conforme a las instrucciones del profesor. Todo el despliegue se documenta de forma analítica.

Entregables del Avance 4

1. `06_aws_setup.py` (modo teórico) – pipeline de configuración teórica de infraestructura AWS.
2. `07_lambda_functions.py` (modo teórico) – funciones Lambda documentadas y comentadas.
3. `aws_architecture_diagram.png` – diagrama de flujo del entorno cloud.
4. `AWS_Análisis_Arquitectura.docx` – documento explicativo de la arquitectura y funcionamiento.